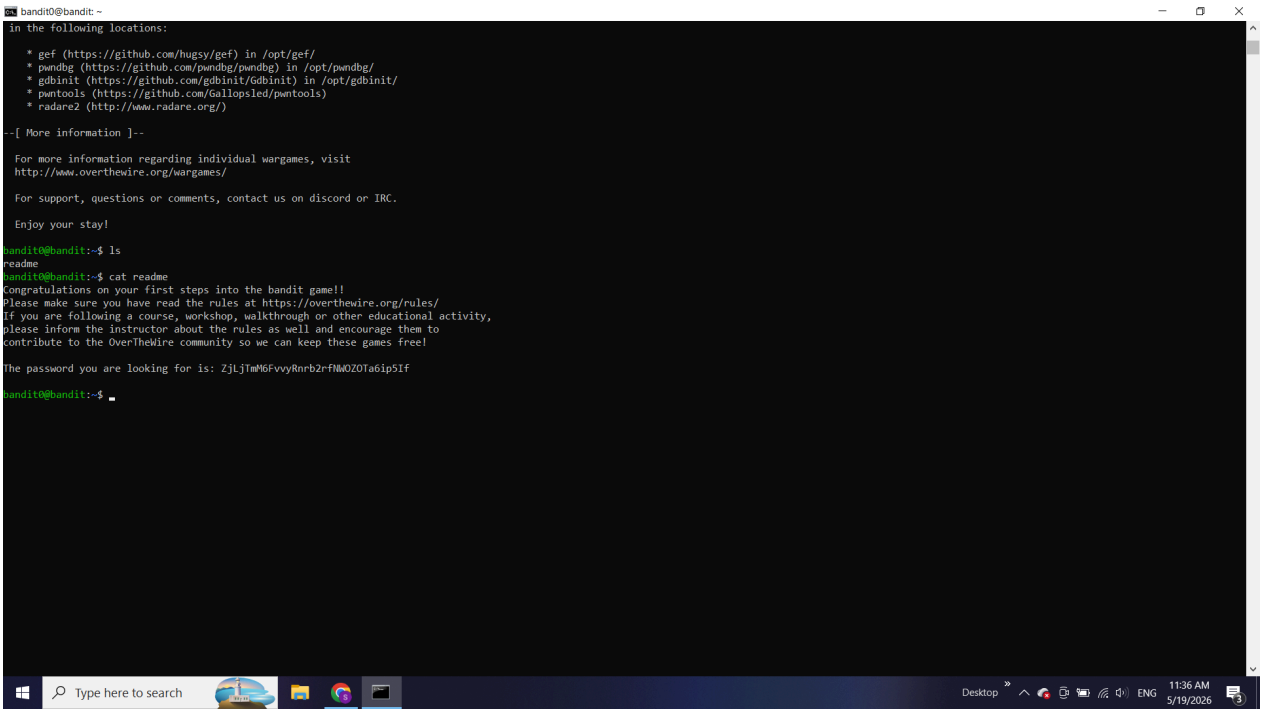
Spinu Sorana Alexandra
Group 1032

GAMES ASSIGNMENT- BANDIT

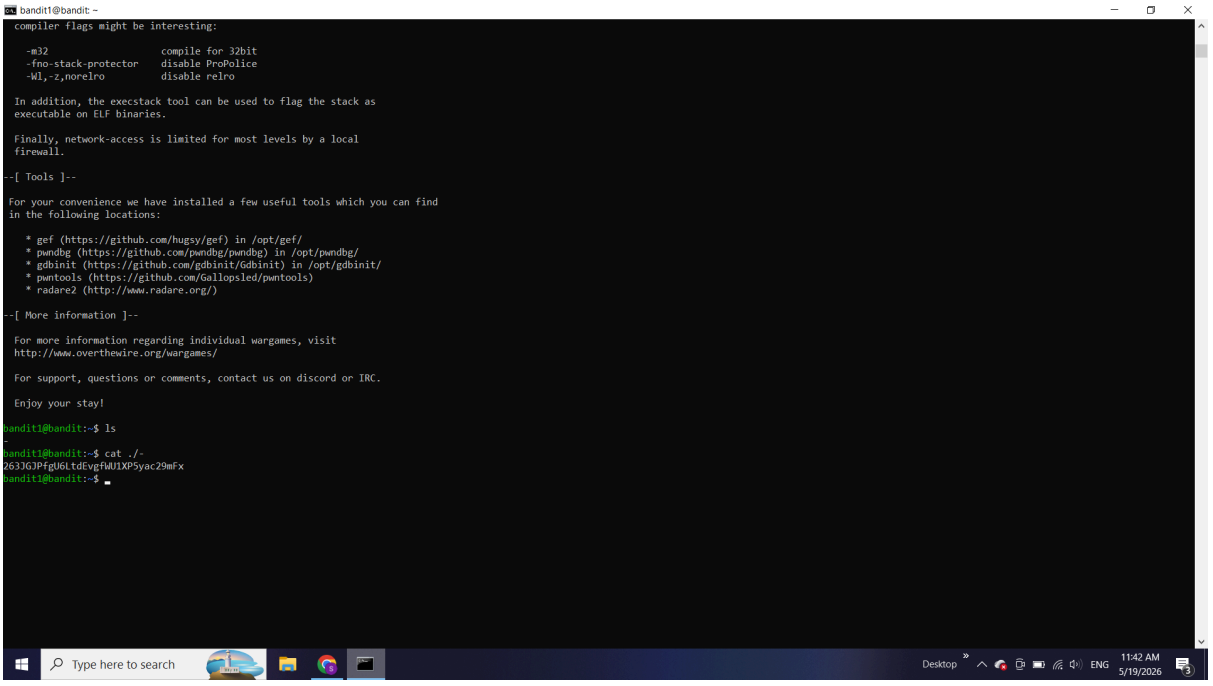
Level 0 - Level 1

- **The Challenge:** The objective was to log into the game server using SSH and retrieve the first password stored in a file named readme located in the home directory.
- **Commands Used:** ssh, ls, cat
- **What I Learned:** I learned how to establish a secure remote connection via SSH on a non-standard port (-p 2220) and how to read the contents of a basic text file using cat.
- **Documentation Proof and solution:**



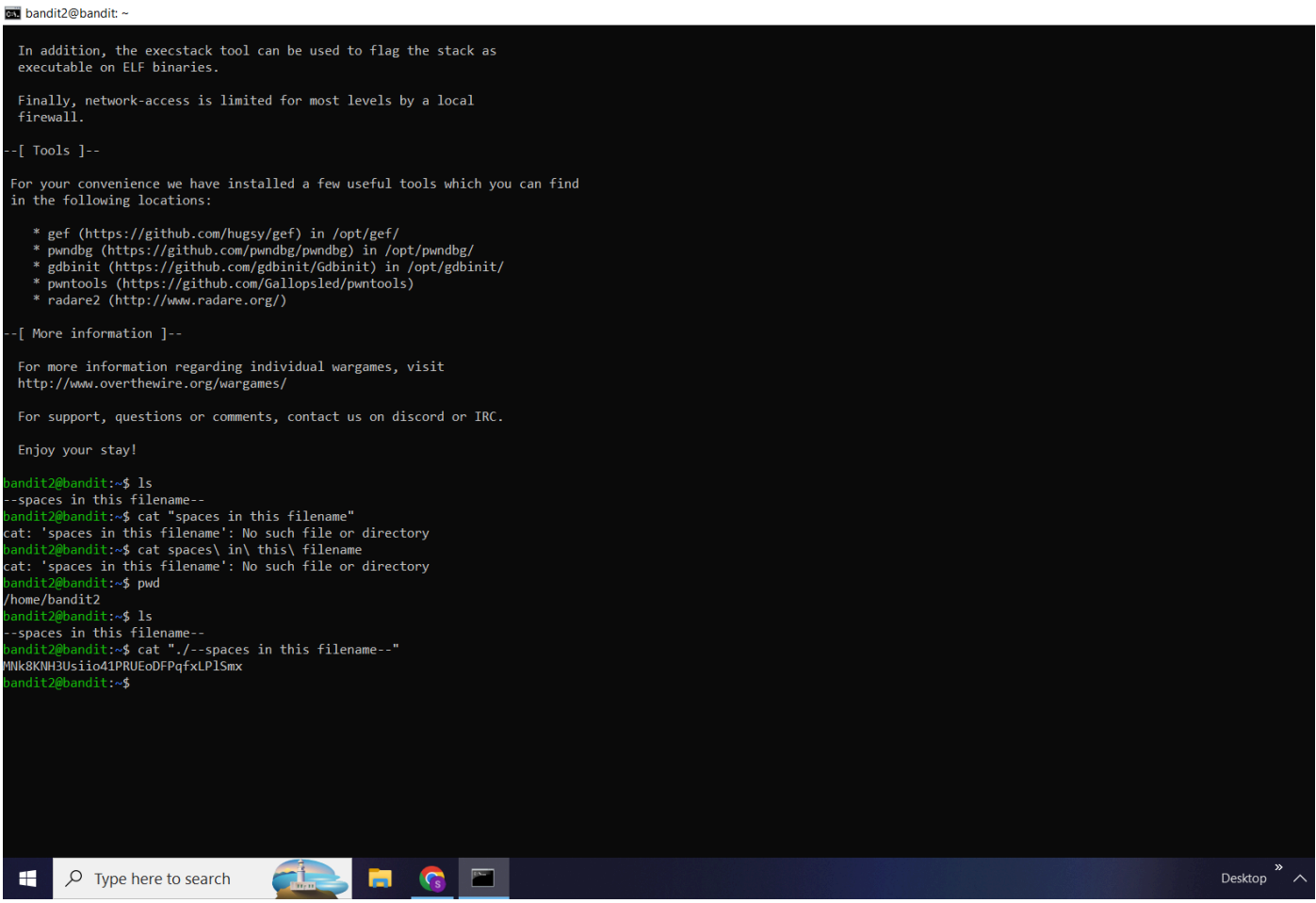
Level 1 - Level 2

- **The Challenge:** The password was hidden in a file named - in the home directory. The dash symbol is usually interpreted by Linux commands as an option indicator, making it difficult to read.
- **Commands Used:** cat
- **What I Learned:** I learned how to bypass command-line option parsing by using the relative path syntax (./-). This explicitly tells the cat command to treat the dash as a filename rather than an argument.
- **Documentation Proof and solution:**



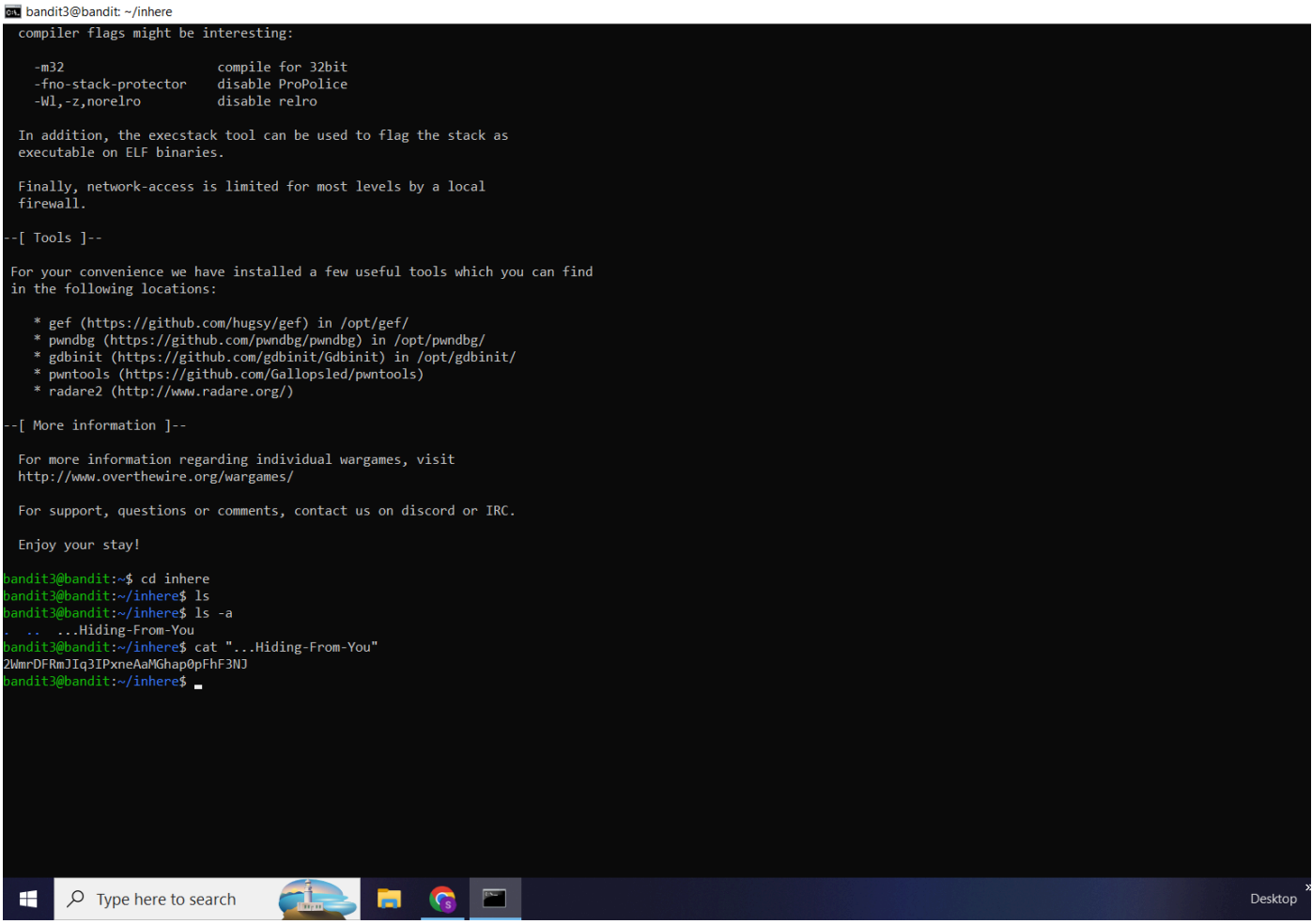
Level 2 - Level 3

- **The Challenge:** The password was stored in a file that contained spaces in its name, which had been updated to --spaces in this filename--. This combined both spaces and leading dashes.
- **Commands Used:** ls, cat
- **What I Learned:** I learned that files with spaces must be enclosed in quotes so the shell treats them as a single argument. Additionally, I combined this with the relative path ("./--") to handle the leading dashes.
- **Documentation Proof and solution:**



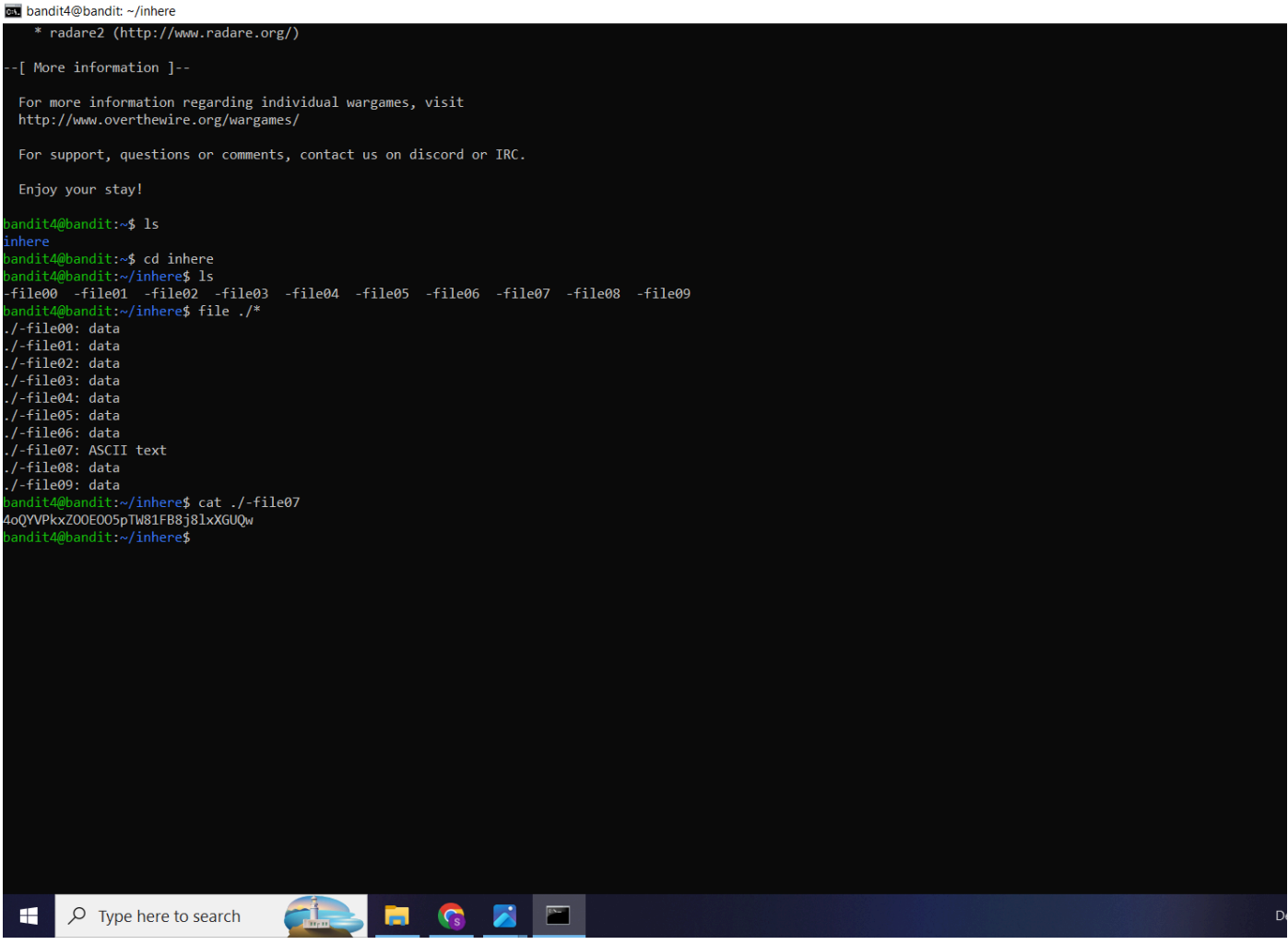
Level 3 - Level 4

- **The Challenge:** The password was hidden in a secret file inside the inhere directory. The file was invisible to a normal directory listing.
- **Commands Used:** cd, ls, ls -a, cat
- **What I Learned:** I learned about hidden files in Linux (which always start with a dot .). I discovered how to use ls -a to list all files, including hidden ones, and successfully read the nested file ...Hiding-From-You.
- **Documentation Proof:**



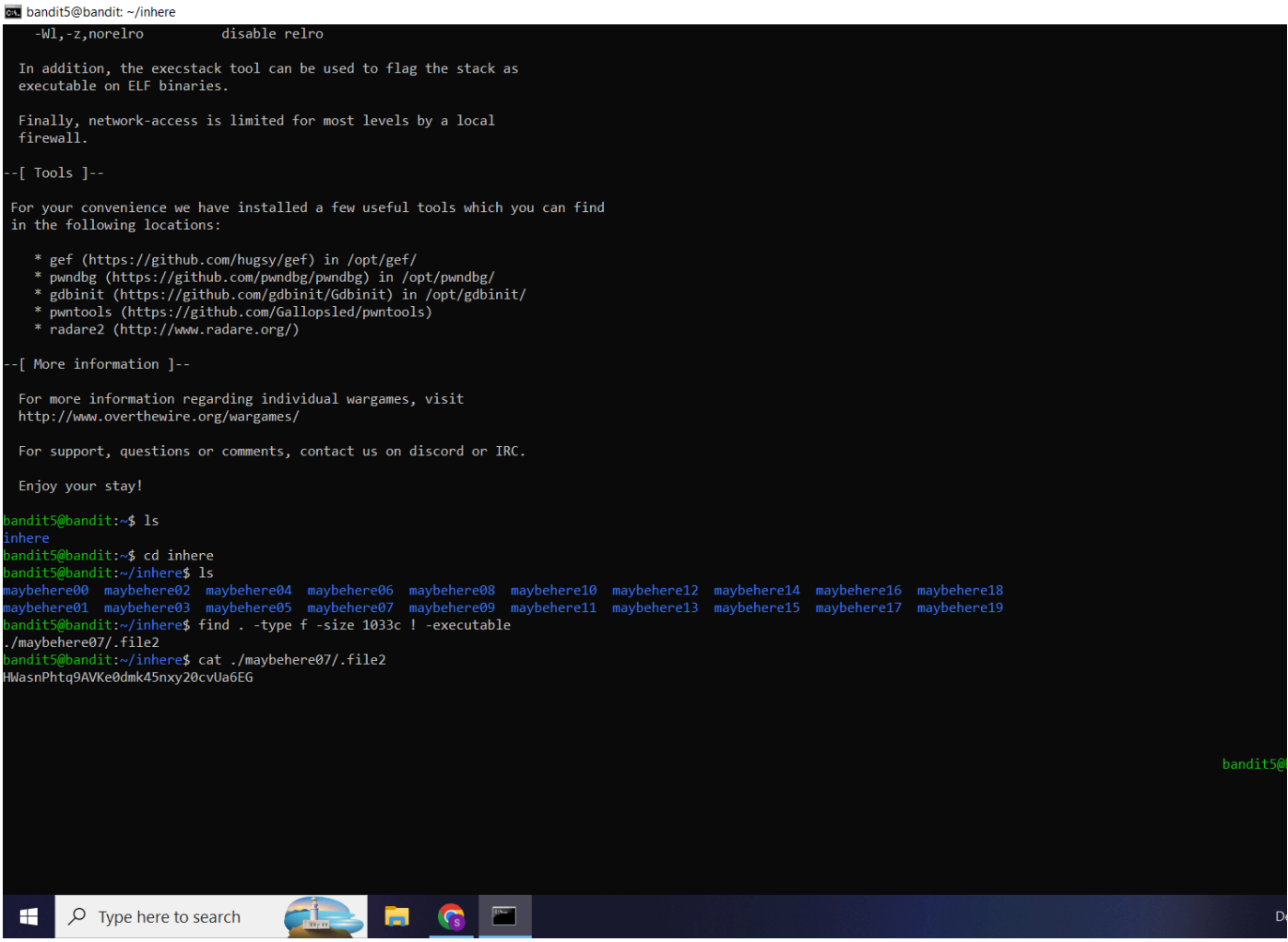
Level 4 - Level 5

- **The Challenge:** The objective was to find the only human-readable text file inside a directory containing ten look-alike files named from -file00 to -file09.
- **Commands Used:** cd, file, cat
- **What I Learned:** I learned how to use the file command combined with a wildcard (./*) to inspect file headers. This allowed me to immediately identify the single "ASCII text" file without messing up the terminal with binary data.
- **Documentation Proof:**



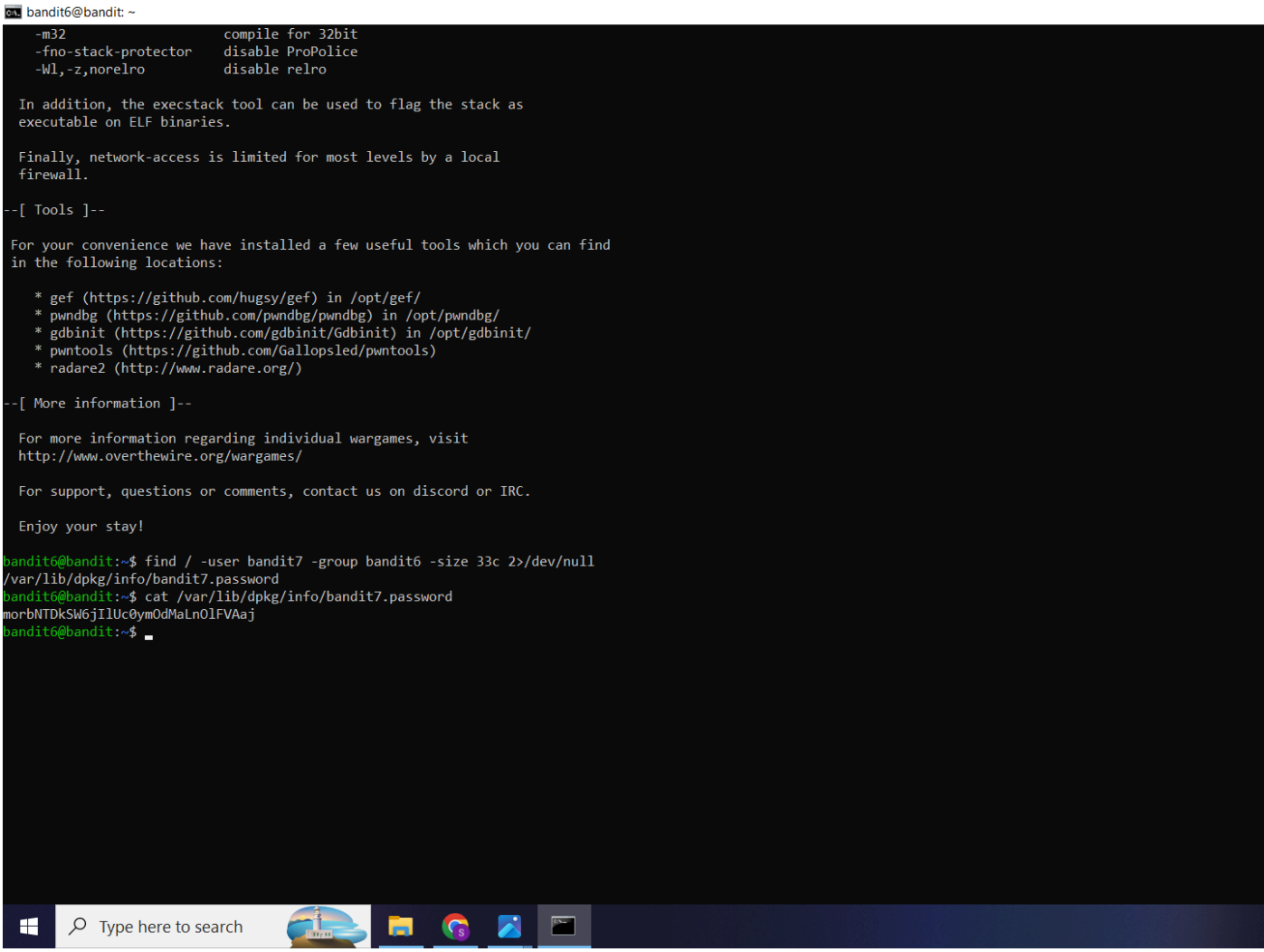
Level 5 - Level 6

- **The Challenge:** The password was hidden within a deeply nested structure of multiple directories. The target file matched specific attributes: human-readable, exactly 1033 bytes, and non-executable.
- **Commands Used:** find, cat
- **What I Learned:** I learned how to leverage advanced criteria within the find utility using flags like -size, -type f, and negation (! -executable) to instantly isolate a single file out of thousands.
- **Documentation Proof:**



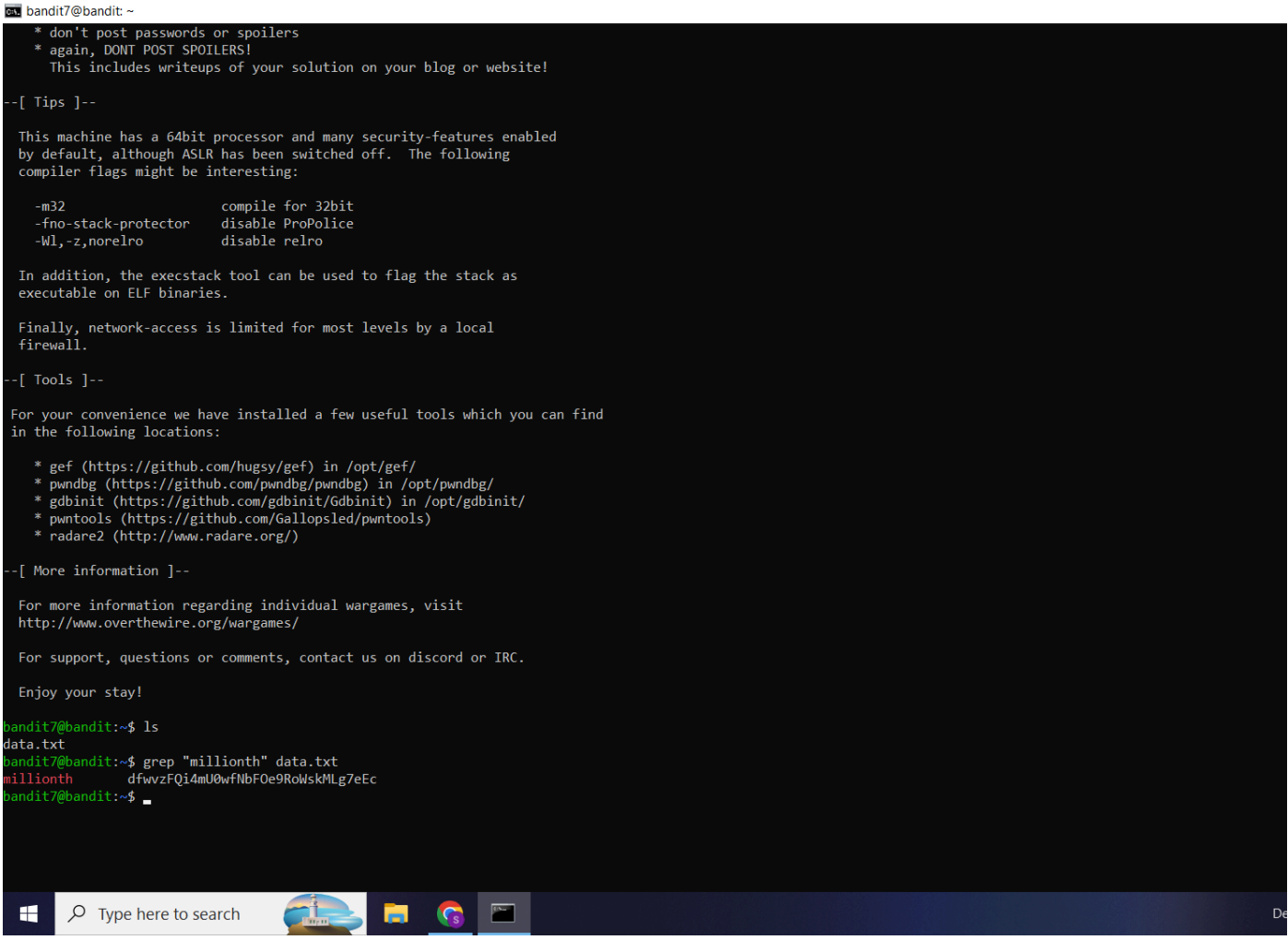
Level 6 - Level 7

- **The Challenge:** The password file was stored somewhere on the entire server, outside the home directory. It belonged to user bandit7, group bandit6, and was exactly 33 bytes in size.
- **Commands Used:** find, cat
- **What I Learned:** I learned how to perform a global system search from the root directory (/). I also learned how to use error redirection (2>/dev/null) to suppress "Permission denied" messages and isolate the valid result.
- **Documentation Proof:**


bandit6@bandit: ~
-m32 compile for 32bit
-fno-stack-protector disable ProPolice
-Wl,-z,norelro disable relro
In addition, the execstack tool can be used to flag the stack as
executable on ELF binaries.
Finally, network-access is limited for most levels by a local
firewall.
--[Tools]--
For your convenience we have installed a few useful tools which you can find
in the following locations:
* gef (https://github.com/hugsy/gef) in /opt/gef/
* pwndbg (https://github.com/pwndbg/pwndbg) in /opt/pwndbg/
* gdbinit (https://github.com/gdbinit/Gdbinit) in /opt/gdbinit/
* pwntools (https://github.com/Gallopsled/pwntools)
* radare2 (http://www.radare.org/)
--[More information]--
For more information regarding individual wargames, visit
http://www.overthewire.org/wargames/
For support, questions or comments, contact us on discord or IRC.
Enjoy your stay!
bandit6@bandit:~$ find / -user bandit7 -group bandit6 -size 33c 2>/dev/null
/var/lib/dpkg/info/bandit7.password
bandit6@bandit:~$ cat /var/lib/dpkg/info/bandit7.password
morbNTDkSW6jIlUc0ymOdMaLnOlFVAaj
bandit6@bandit:~$
Type here to search

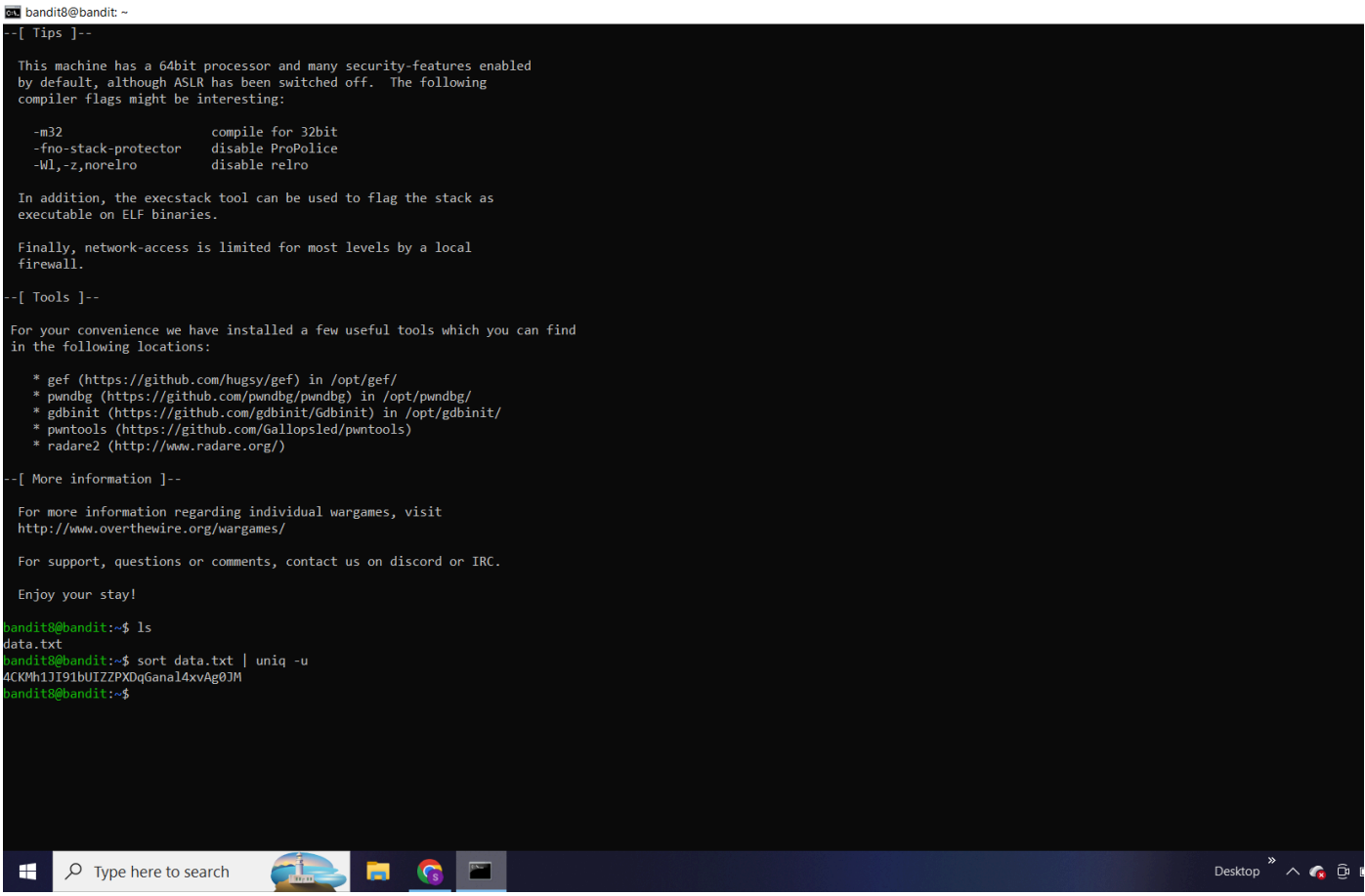
Level 7 - Level 8

- **The Challenge:** The password was stored in a massive text file called data.txt, next to the word "millionth".
- **Commands Used:** ls, grep
- **What I Learned:** I learned how to extract specific information from large text bases using grep. Instead of reading millions of lines, grep scanned the file and printed only the line matching the specific keyword.
- **Documentation Proof:**



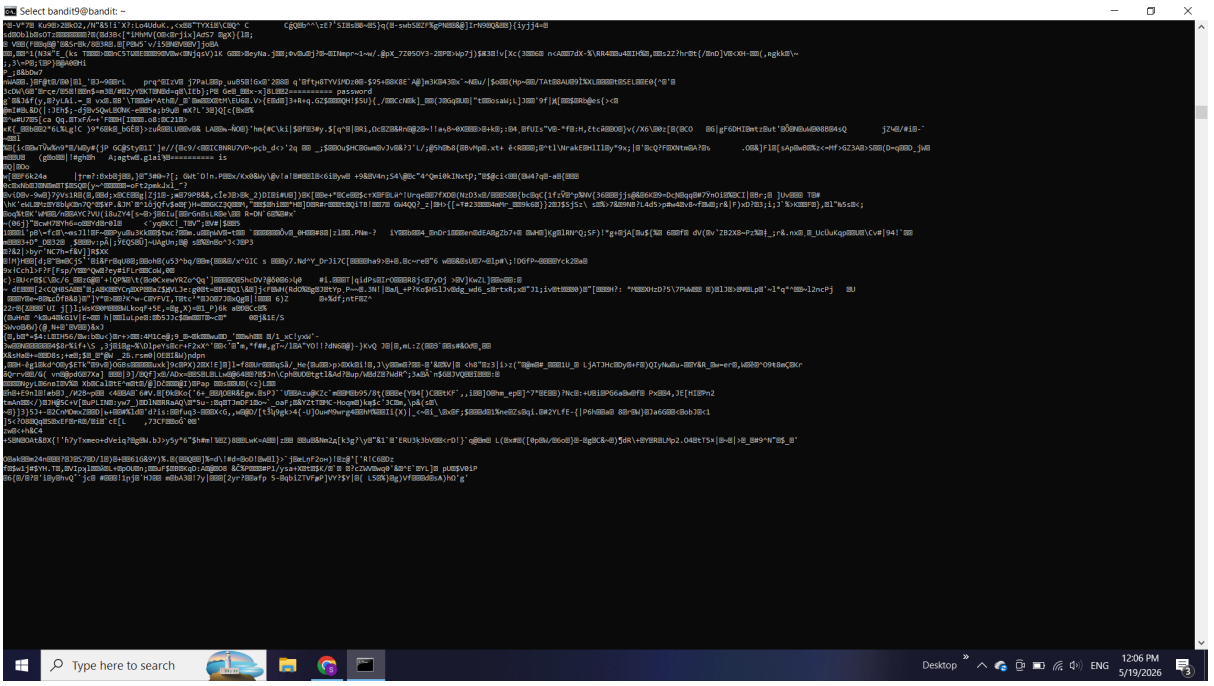
Level 8 - Level 9

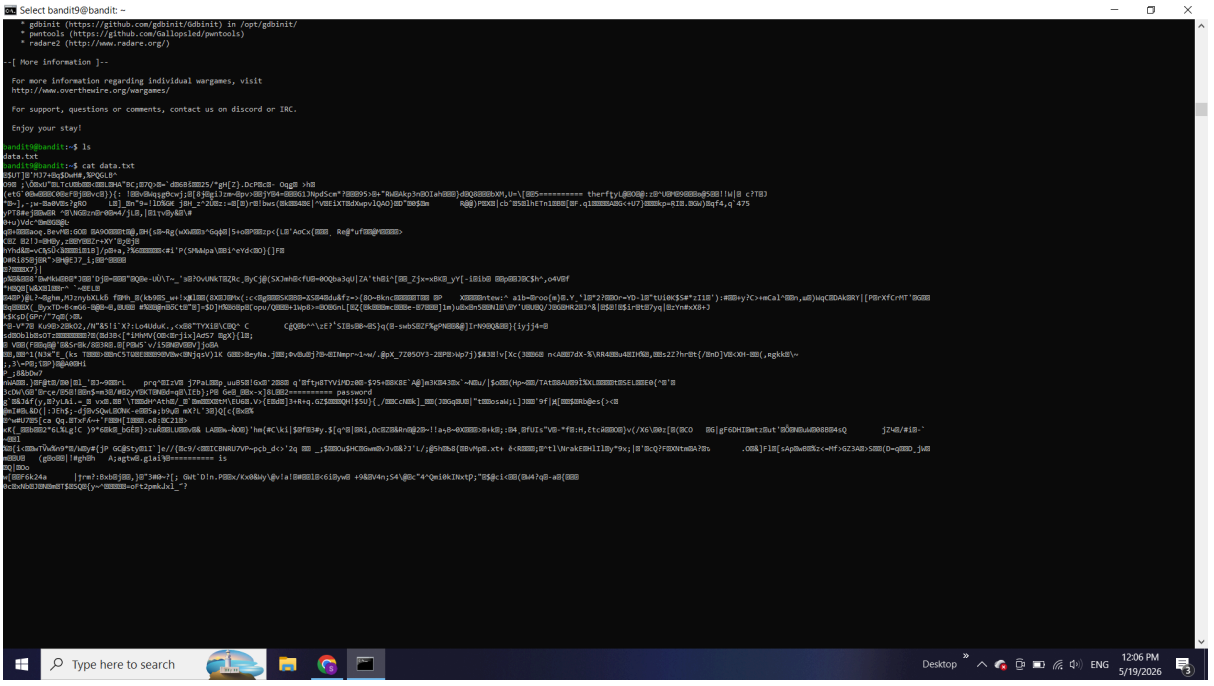
- **The Challenge:** The password was stored in data.txt and was defined as the only line of text that occurs exactly once. All other lines were repeated multiple times.
- **Commands Used:** sort, uniq
- **What I Learned:** I learned how to use the pipeline operator (|) to combine utilities. I sorted the text using sort (a prerequisite for uniq) and filtered out all duplicates using uniq -u to find the unique line.
- **Documentation Proof:**

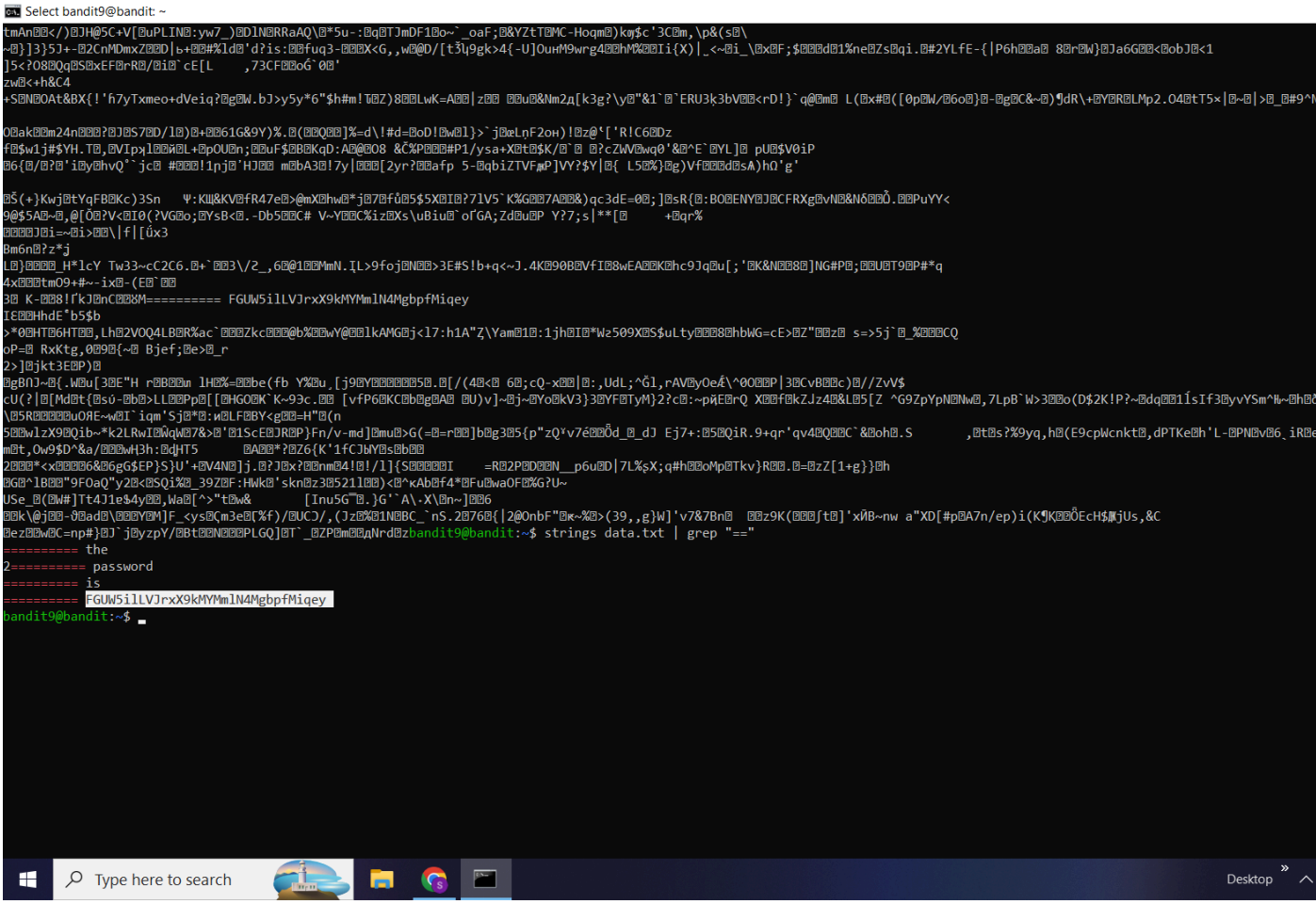


Level 9 - Level 10

- **The Challenge:** The password was hidden inside a binary file (data.txt). The password string was surrounded by several = characters.
- **Commands Used:** strings, grep
- **What I Learned:** I learned how to parse binary data to extract human-readable text using the strings command. By piping the output to grep "==", I successfully filtered out garbage data and located the password.
- **Documentation Proof:**






Select bandit9@bandit: ~
bandit9@bandit:~$ strings data.txt | grep "=="
========== the
2========== password
========== is
========== FGUW5ilLVJrxX9kMYMmlN4MgbpfMiqey
bandit9@bandit:~$
Type here to search
Desktop

Level 10 - Level 11

- The Challenge: The password was stored in data.txt but was encoded in Base64 format, making it unreadable in its raw form.
- Commands Used: base64
- **What I Learned:** I learned about data encoding standards and how to decode Base64 data directly from the Linux command line using the base64 -d utility to reveal the final plain-text password.
- **Documentation proof:**

